Hw7

zhijing huang

2026-10-11

Question 1

a. The prior probabilities are listed as follows in class 0 and class 1, along with the means and covariance matrix.

```
heart <- read.csv("/Users/zhijinghuang/Downloads/Stat432_2026/week-07/data/saheart.csv")
heart$famhist <- factor(heart$famhist, levels = c("Absent", "Present"))
X <- data.matrix(heart[, names(heart) != "chd"])
y <- heart$chd

n <- nrow(X)
n0 <- sum(y == 0)
n1 <- sum(y == 1)
prior0 <- n0 / n
prior1 <- n1 / n
c(Class0 = prior0, Class1 = prior1)
```

```
##    Class0    Class1
## 0.6536797 0.3463203
```

```
X0 <- X[y == 0, , drop = FALSE]
X1 <- X[y == 1, , drop = FALSE]
mu0 <- colMeans(X0)
mu1 <- colMeans(X1)

rbind(Class0 = mu0, Class1 = mu1)
```

```
##             sbp  tobacco      ldl adiposity  famhist    typea  obesity  alcohol
## Class0 135.4603 2.634735 4.344238  23.96911 1.317881 52.36755 25.73745 15.93136
## Class1 143.7375 5.524875 5.487938  28.12025 1.600000 54.49375 26.62294 19.14525
##             age
## Class0 38.85430
## Class1 50.29375
```

```
#pooled cov matrix
S0 <- cov(X0)
S1 <- cov(X1)
Sigma_pool <- (
  (n0 - 1) * S0 +
  (n1 - 1) * S1
) / (n - 2)

Sigma_pool
```

```
##                   sbp    tobacco         ldl  adiposity    famhist        typea
## sbp       405.4347786 14.5851882  4.58124834 49.1643582 0.33698819 -15.58772153
## tobacco    14.5851882 19.2425564  0.76320271  7.5381093 0.01582050  -2.05730592
## ldl         4.5812483  0.7632027  4.00058129  6.0326970 0.09185024   0.34459233
## adiposity  49.1643582  7.5381093  6.03269701 56.7528931 0.43280832  -5.30956633
## famhist     0.3369882  0.0158205  0.09185024  0.4328083 0.22583357   0.08112007
## typea     -15.5877215 -2.0573059  0.34459233 -5.3095663 0.08112007  95.56564291
## obesity    18.9388430  1.8334506  2.66005159 22.7078852 0.18402776   2.64006728
## alcohol    64.4003517 20.5170451 -2.53290675 16.1190253 0.76846885   7.96003164
## age        95.1338895 22.7654113  6.47892049 60.5086715 0.99736251 -20.27833492
##               obesity     alcohol         age
## sbp       18.9388430  64.4003517  95.1338895
## tobacco    1.8334506  20.5170451  22.7654113
## ldl        2.6600516  -2.5329068   6.4789205
## adiposity 22.7078852  16.1190253  60.5086715
## famhist    0.1840278   0.7684688   0.9973625
## typea      2.6400673   7.9600316 -20.2783349
## obesity   17.6154236   4.6893563  15.6969995
## alcohol    4.6893563 598.2766133  27.8859413
## age       15.6969995  27.8859413 184.1321373
```

b.

```
w0 <- solve(Sigma_pool, mu0)
w1 <- solve(Sigma_pool, mu1)
score0 <- as.numeric( X %*% w0 - 0.5 * sum(mu0 * w0) + log(prior0))
score1 <- as.numeric( X %*% w1 - 0.5 * sum(mu1 * w1) + log(prior1))
head(data.frame( Actual = y, Score0 = score0, Score1 = score1))
```

```
##   Actual   Score0   Score1
## 1      1 64.91051 65.93107
## 2      1 66.06673 65.18497
## 3      0 54.66488 53.66428
## 4      1 73.07422 74.04532
## 5      1 59.66606 60.55175
## 6      0 64.00305 64.50967
```

```
pred_manual <- ifelse(score1 > score0, 1, 0)

confusion_manual <- table(
  Observed = factor(y, levels = c(0, 1)),
  Predicted = factor(pred_manual, levels = c(0, 1))
)

confusion_manual
```

```
##          Predicted
## Observed   0   1
##        0 258  44
##        1  73  87
```

c. After calculation, the results do match with each other.

```
fit_lda <- MASS::lda(
  x = X,
  grouping = factor(y, levels = c(0, 1)),
```

```
  prior = c(prior0, prior1),
  method = "moment"
)

pred_package <- predict(fit_lda, newdata = X)$class
confusion_package <- table(
  Observed = factor(y, levels = c(0, 1)),
  Predicted = pred_package
)

confusion_package
```

```
##          Predicted
## Observed   0   1
##        0 258  44
##        1  73  87
```

Question 2

a. The misclassification rate is 0.1554559.

```
digits_train <- read.csv(gzfile("/Users/zhijinghuang/Downloads/Stat432_2026/week-07/data/zip-train.csv.gz"),
digits_test <- read.csv(gzfile("/Users/zhijinghuang/Downloads/Stat432_2026/week-07/data/zip-test.csv.gz"), h

X_train <- as.matrix(digits_train[, -1])
X_test <- as.matrix(digits_test[, -1])
y_train <- factor(digits_train[, 1], levels = 0:9)
y_test <- factor(digits_test[, 1], levels = 0:9)

ncol(X_train)
```

```
## [1] 256
```

```
table(y_train)
```

```
## y_train
##    0    1    2    3    4    5    6    7    8    9
## 1194 1005  731  658  652  556  664  645  542  644
```

```
pixel_variances <- apply(X_train, 2, var)

pixel_order <- order(pixel_variances, decreasing = TRUE)
selected40 <- pixel_order[1:40]

data.frame(
  Pixel = selected40,
  Training_variance = pixel_variances[selected40]
)
```

```
##      Pixel Training_variance
## V231   230         0.7995005
## V220   219         0.7786657
## V106   105         0.7761626
## V186   185         0.7760059
```

```
## V122   121         0.7698937
## V121   120         0.7651751
## V205   204         0.7642945
## V57     56         0.7634517
## V236   235         0.7614990
## V137   136         0.7610922
## V170   169         0.7609557
## V221   220         0.7572460
## V214   213         0.7567441
## V138   137         0.7499158
## V77     76         0.7443015
## V202   201         0.7433579
## V28     27         0.7427205
## V39     38         0.7414070
## V70     69         0.7394806
## V23     22         0.7391538
## V201   200         0.7385493
## V105   104         0.7379827
## V215   214         0.7365760
## V154   153         0.7365185
## V153   152         0.7356207
## V54     53         0.7354114
## V108   107         0.7342641
## V124   123         0.7341245
## V61     60         0.7334178
## V55     54         0.7315470
## V58     57         0.7308114
## V235   234         0.7302140
## V93     92         0.7301541
## V92     91         0.7284644
## V76     75         0.7264357
## V140   139         0.7255123
## V155   154         0.7226698
## V185   184         0.7214531
## V169   168         0.7213691
## V44     43         0.7213038
```

```
X_train40 <- X_train[, selected40, drop = FALSE]
X_test40 <- X_test[, selected40, drop = FALSE]
fit40 <- MASS::qda(
  x = X_train40,
  grouping = y_train
)

pred40 <- predict(fit40, newdata = X_test40)$class
confusion40 <- table( Observed = y_test, Predicted = factor(pred40, levels = 0:9))
error40 <- mean( as.character(pred40) != as.character(y_test))

confusion40
```

```
##          Predicted
## Observed   0   1   2   3   4   5   6   7   8   9
##        0 319   0  24   6   2   2   1   0   5   0
##        1   0 238   5   0  11   0   7   0   3   0
##        2   8   0 168   8   8   3   0   0   3   0
##        3   5   0   4 128   0  14   0   0  15   0
##        4   0   2  13   1 150   1   0   1   7  25
##        5  12   0   3   5   1 131   0   0   6   2
```

```
##         6   0   0   8   0   1   8 148   0   5   0
##         7   1   0   6   1   6   0   0 124   2   7
##         8   1   0  10  12   3   2   0   0 137   1
##         9   0   0   1   2   7   1   1   4   9 152
```

```
error40
```

```
## [1] 0.1554559
```

b. The method worked, because it resulted in a confusion matrix where most observations are diagonal. Reducing the number of predictors helped avoid the within-class covariance singularity encountered with all 256 pixels. Selecting pixels with high training variance also avoids nearly constant pixels overall, although it does not guarantee that the selected predictors are informative or that every within-class covariance matrix is nonsingular.

c. The misclassification rate is 0.3423019. The 10 predictor model didn't turn out to work as well as the 40 predictor model, as the observations are more scattered around the matrix, instead of forming an obvious diagonal trend. Reducing the number of pixels makes the covariance matrices easier to estimate, but it also removes potentially useful information for distinguishing digits.

```
selected10 <- pixel_order[1:10]

X_train10 <- X_train[, selected10, drop = FALSE]
X_test10 <- X_test[, selected10, drop = FALSE]

fit10 <- MASS::qda(
  x = X_train10,
  grouping = y_train
)

pred10 <- predict(fit10, newdata = X_test10)$class

confusion10 <- table(
  Observed = y_test,
  Predicted = factor(pred10, levels = 0:9)
)

error10 <- mean(
  as.character(pred10) != as.character(y_test)
)

confusion10
```

```
##          Predicted
## Observed   0   1   2   3   4   5   6   7   8   9
##        0 298   0  11   1   1  16  28   4   0   0
##        1   0 241  10   0   3   1   1   0   2   6
##        2  19   2  95  14  14   3  22  21   5   3
##        3   5   0   6 103   3  42   0   0   7   0
##        4   1   5  14   0  95   2   4  15  17  47
##        5  15   0   6  25   4  96   4   2   8   0
##        6  41   0  15   4   0  18  78   7   5   2
##        7   4   1   6   0   6   0   1 122   0   7
##        8   1   2  11  36  12  11   4   1  74  14
##        9   0   2   9   2  20   0   0  16  10 118
```

```
error10
```

```
## [1] 0.3423019
```

Question 3

a.

```
set.seed(43207)
rho <- c(0, 0.2, 0.4, 0.6, 0.8, 0.9)
p <- 10
run_experiment <- function(rho) {
  mu1 <- c(-1, rep(0, p - 1))
  mu2 <- c(1, rep(0, p - 1))
  Sigma1 <- diag(p)
  Sigma2 <- diag(p)

  Sigma1[1, 2] <- Sigma1[2, 1] <- rho
  Sigma2[1, 2] <- Sigma2[2, 1] <- -rho

  train1 <- MASS::mvrnorm( n = 100, mu = mu1, Sigma = Sigma1)
  train2 <- MASS::mvrnorm( n = 100, mu = mu2, Sigma = Sigma2)

  test1 <- MASS::mvrnorm( n = 1000, mu = mu1, Sigma = Sigma1)
  test2 <- MASS::mvrnorm( n = 1000, mu = mu2, Sigma = Sigma2)

  X_train <- rbind(train1, train2)
  X_test <- rbind(test1, test2)
  y_train <- factor(rep(c(1, 2), each = 100))
  y_test <- factor(rep(c(1, 2), each = 1000))

  fit_lda <- MASS::lda( x = X_train, grouping = y_train, prior = c(0.5, 0.5))
  fit_qda <- MASS::qda(x = X_train, grouping = y_train, prior = c(0.5, 0.5))
  pred_lda <- predict(fit_lda, newdata = X_test)$class
  pred_qda <- predict(fit_qda, newdata = X_test)$class
  c(LDA = mean(pred_lda != y_test), QDA = mean(pred_qda != y_test))
}
mu1 <- c(-1, rep(0, p - 1))
single_errors <- matrix(
  NA_real_,
  nrow = length(rho),
  ncol = 2,
  dimnames = list(NULL, c("LDA", "QDA"))
)
for (j in seq_along(rho)) {
  single_errors[j, ] <- run_experiment(rho[j])
}
results_a <- data.frame(
  rho = rho,
  LDA_error = single_errors[, "LDA"],
  QDA_error = single_errors[, "QDA"]
)

results_a
```

```
##   rho LDA_error QDA_error
```

```
## 1 0.0    0.1650    0.1885
## 2 0.2    0.1765    0.1920
## 3 0.4    0.1745    0.2105
## 4 0.6    0.1720    0.1800
## 5 0.8    0.1710    0.1415
## 6 0.9    0.1775    0.1030
```

b.

```
B <- 50

average_errors <- matrix(
  NA_real_,
  nrow = length(rho),
  ncol = 2,
  dimnames = list(NULL, c("LDA", "QDA"))
)

for (j in seq_along(rho)) {
  repeated_errors <- matrix(
    NA_real_,
    nrow = B,
    ncol = 2
  )

  for (b in seq_len(B)) {
    repeated_errors[b, ] <- run_experiment(rho[j])
  }

  average_errors[j, ] <- colMeans(repeated_errors)
}
results_b <- data.frame(
  rho = rho,
  LDA_average_error = average_errors[, "LDA"],
  QDA_average_error = average_errors[, "QDA"]
)

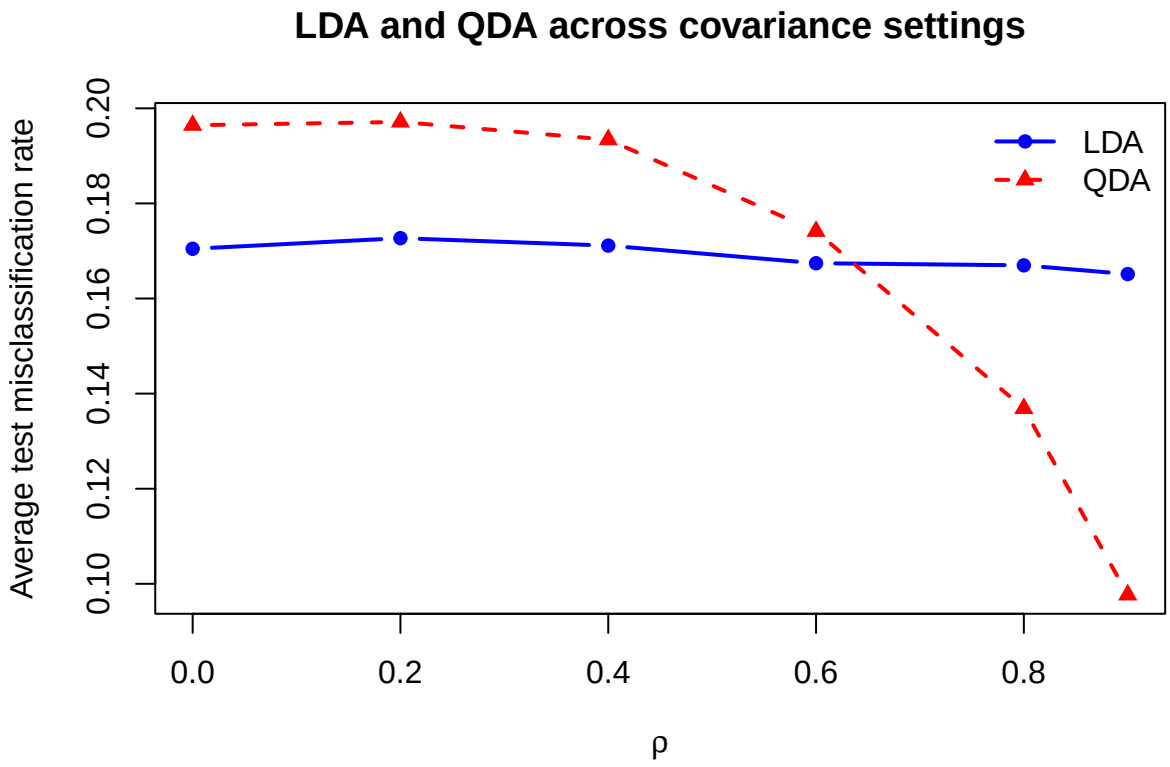
results_b
```

```
##   rho LDA_average_error QDA_average_error
## 1 0.0           0.17046           0.19644
## 2 0.2           0.17269           0.19713
## 3 0.4           0.17113           0.19341
## 4 0.6           0.16742           0.17413
## 5 0.8           0.16698           0.13688
## 6 0.9           0.16514           0.09768
```

```
matplot(
  rho,
  average_errors,
  type = "b",
  pch = c(16, 17),
  lty = c(1, 2),
  col = c("blue", "red"),
  lwd = 2,
  xlab = expression(rho),
  ylab = "Average test misclassification rate",
```

```
  main = "LDA and QDA across covariance settings"
)

legend(
  "topright",
  legend = c("LDA", "QDA"),
  col = c("blue", "red"),
  pch = c(16, 17),
  lty = c(1, 2),
  lwd = 2,
  bty = "n"
)
```



c. The true covariance matrixs are identical at rho = 0, therefore, it is appropriate to use LDA's assumption. QDA estimates two matrices even though the underlying matrices are the same, making it weak in small rho values. However, QDA performs better at capturing missing information. At a larger rho, QDA's flexibility may provide enough benefit to outweigh its higher estimation variance. QDA can capture the increasingly different covariance structures, whereas LDA assumes a shared covariance. The eight noise predictors add estimation variability despite providing no information about class membership.

Question 4

a. Skill definition. I used an explicitly invoked skill for comparing LDA and QDA. The skill file is 1,790 bytes, below the 5,000-byte limit. Its complete contents appear below.

b. Application. I invoked the skill with this prompt: "Use the skill in `week-07/lda-qda-analysis/SKILL.md` to analyze `week-07/data/saheart.csv`. Predict `chd` using the nine other variables, coding `famhist` as Absent = 1 and Present = 2. Compare ordinary LDA and QDA and seek the best prediction performance within the skill's allowed methods. Report the seed, cross-validation results, and reserved test error."

I used seed 432074 and a stratified 70/30 split: 323 training and 139 test observations. Eight candidates included ordinary LDA and QDA, plus each method after PCA with 3, 5, or 7 components. I compared them using five shared stratified cross-validation folds, fitting PCA scaling and directions separately in each fold. PCA with five components followed by QDA had the lowest cross-validation error (0.291). After refitting on all 323 training observations, it made 32 errors on the reserved test set (0.230 misclassification rate). The one-page analysis report is appended after the skill.

c. Review and submission. I reviewed the results without using the test set to change candidates. The test set is small enough that minor performance differences should not be treated as conclusive. The file is outside automatically loaded skill folders; I supplied its path explicitly in the invocation prompt.

Complete `SKILL.md`:

```
---
name: lda-qda-analysis
description: >-
  Use only when explicitly invoked to compare LDA and QDA
  classifiers and produce a concise statistical report.
---

# LDA/QDA analysis

Ask for the dataset, response, and predictor set if they are not
supplied. Do not assume a particular dataset or variable name. Use
LDA, QDA, or combinations of their predicted class probabilities;
a representation such as PCA is allowed before either classifier.

Reserve a test set before comparing candidates. Within the remaining
training data, use the same stratified cross-validation folds for
every candidate. Fit every transformation, including imputation,
scaling, and PCA, using only the fitting portion of each fold. Apply
its fitted parameters to that fold's validation observations. Select
candidate settings using mean cross-validation misclassification
rate, with a stated tie rule. Never use test results to revise
candidates or settings.

Refit the selected transformation and classifier on all training
observations. Predict the reserved test set once and report its
misclassification rate, number of errors, and sample size. Include
ordinary LDA and QDA as comparison candidates. Check class counts
and covariance rank before fitting QDA; report failures rather than
substituting another classifier silently. Keep the random seed,
folds, and candidate settings reproducible.

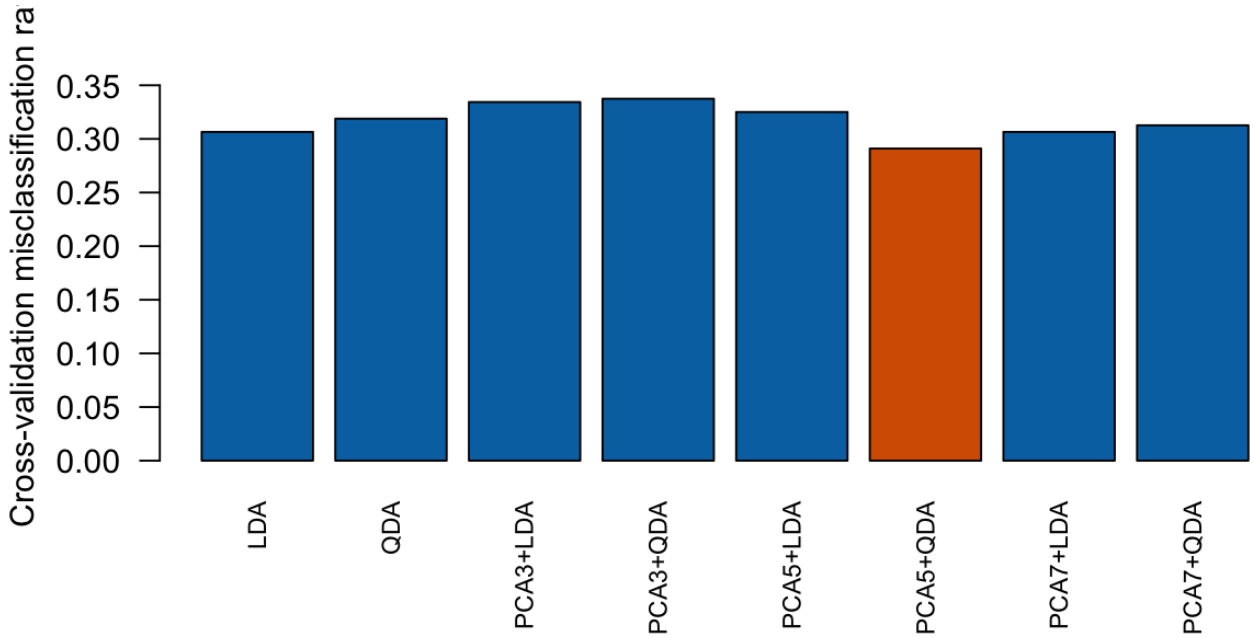
Produce a clearly readable analysis report of at most one printed
page. Include a figure comparing validation performance, a compact
table of candidate results, and numerical results for the selected
model on the test set. Briefly state the candidates, validation
design, selected model, main finding, and limitations. Use a
model-based explanation rather than claiming a tiny difference
establishes superiority.
```

The one-page analysis report is appended as the final page of the submitted PDF.

SAheart: LDA/QDA model comparison

Seed 432074 | 70/30 stratified split | Five shared stratified CV folds
Response: chd | Nine numeric predictors; famhist coded Absent=1, Present=2

Candidates and validation results



Model	CV error	Fold SD
LDA	0.307	0.045
QDA	0.319	0.070
PCA3+LDA	0.334	0.040
PCA3+QDA	0.337	0.054
PCA5+LDA	0.325	0.058
PCA5+QDA (selected)	**0.291**	**0.061**
PCA7+LDA	0.307	0.059
PCA7+QDA	0.313	0.047

Selected fit and reserved test set

PCA5+QDA was refitted on all 323 training cases.
Test error: 32/139 = 0.230 (23.0%).
PCA scaling and directions were re-estimated within each CV fitting fold.

Finding and limitation

PCA5+QDA had the lowest CV error. Its test error was lower, but the test set contains only 139 cases; small model differences remain uncertain.